PROJECT REPORT

LearnLoop: Adapting an Open-Source Agentic AI Tutor for Local Deployment

[Your Name]
[Department, College] · [email address]
November 2026

***Abstract*—Large language models (LLMs) make it plausible to give every learner a patient tutor that is always available, but most AI tutors are hosted services whose data, cost and availability the learner does not control. This report presents LearnLoop, a locally deployed adaptation of DeepTutor [1], an open-source, agent-native tutoring system from the Data Intelligence Lab at The University of Hong Kong. We make three contributions. First, we analyse the architecture of the system as it exists in code: a durable turn service shared by three entry points, a two-layer plugin model that separates single-shot tools from multi-stage capabilities, a typed streaming event protocol, pluggable retrieval-augmented knowledge bases, a three-layer learner memory and a recency-weighted mastery model. Second, we describe how we adapted the system for a self-contained installation on Windows and macOS: we removed 306 files that a local installation does not use (deployment, desktop packaging, release automation and upstream documentation), renamed the user-facing product through a consistent translation-key transform, and added one-step setup and start scripts. Third, we evaluate the adaptation by regression testing against the unmodified upstream release on the same machine: the remaining 12,774 Python tests and 2,560 frontend tests give the same results as before, with no new failures, and the production web build and an end-to-end conversation succeed. We close with the limitations of this work, notably the absence of a learner study, and propose an evaluation protocol.**

***Index Terms*—intelligent tutoring systems, large language models, LLM agents, retrieval-augmented generation, learner modelling, local deployment.**

1. Introduction

One-to-one tutoring is among the most effective forms of instruction. Bloom reported that students tutored individually performed about two standard deviations above students taught conventionally, and posed the search for methods of group instruction as effective as tutoring as the "2 sigma problem" [2]. Intelligent tutoring systems (ITS) have closed part of that gap: in VanLehn's review, step-based tutoring systems were nearly as effective as human tutors [3]. LLMs have renewed interest in the problem because they can hold an open-ended dialogue about almost any subject [10].

An LLM on its own is not yet a tutor. It can state falsehoods fluently, it cannot see the learner's own course material, it forgets everything between sessions, and it has no model of what the learner already knows. Recent systems address these gaps by letting the model reason in steps and act through tools [6]–[8], by grounding answers in retrieved documents [5], [9], and by keeping persistent state about the learner. DeepTutor [1] combines these ideas in one open-source system with a web interface, a command line and a Python SDK.

Our goal in this project was practical: run such a system entirely on a student's own laptop, including Windows, under a distinct name suitable for a course project, without breaking any of its functionality. Running locally keeps learning data on the device, allows offline use with a local model server, and leaves the cost of model calls under the learner's control. This report makes three contributions:

- an architectural analysis of DeepTutor's runtime, derived from its source code (Sections 3–5);
- a documented, reproducible adaptation for local deployment, LearnLoop (Section 6);
- a regression-based evaluation showing that the adaptation preserves the system's tested behaviour (Section 7).

Section 2 reviews related work, Section 8 discusses limitations and future work, and Section 9 concludes. We did not change any of DeepTutor's algorithms; Sections 3–5 describe the upstream design.

2. Background and Related Work

2.1 Tutoring and learner modelling

Mastery learning holds that a learner should demonstrate command of a unit before moving on, and Bloom's two-sigma result [2] is its best-known motivation. To decide when a skill is mastered, a tutor needs a learner model. Bayesian Knowledge Tracing (BKT) treats knowledge of each skill as a hidden binary state and updates the probability that it has been learned after every correct or incorrect response, using learn, guess and slip parameters [4]. VanLehn's review [3] attributes much of the effectiveness of tutoring systems to such fine-grained, step-level feedback.

2.2 LLM agents and tool use

Chain-of-thought prompting showed that asking a model for intermediate reasoning steps improves multi-step reasoning [6]. ReAct interleaves such reasoning traces with actions on external tools, so that observations from the environment can correct the model's plan [7]. Toolformer showed that a model can learn when and how to call external APIs [8]. The agent loop at the centre of DeepTutor's chat capability follows this reason-act-observe pattern.

2.3 Retrieval-augmented generation

Retrieval-augmented generation (RAG) pairs a generator with a retriever over a document index, so that answers can draw on and cite external knowledge [5]. LightRAG extends the index with a graph of entities and relations and retrieves at two levels of granularity [9]. Retrieval matters especially in tutoring, where an answer should come from the learner's own textbook rather than from the model's general knowledge.

2.4 LLMs in education

Kasneci et al. survey the opportunities LLMs offer education, such as personalised explanations and generated practice, together with risks including fabricated content, over-reliance, threats to academic integrity and privacy [10]. Several of these risks inform the design choices discussed in Sections 5 and 8.

2.5 DeepTutor

DeepTutor [1] is released under the Apache License 2.0 by the Data Intelligence Lab at The University of Hong Kong. LearnLoop is a modified version of its 1.7.2 release. Apache-2.0 permits modification and redistribution provided that the licence, the copyright notices and a statement of changes are retained; LearnLoop keeps all three (Section 6.3).

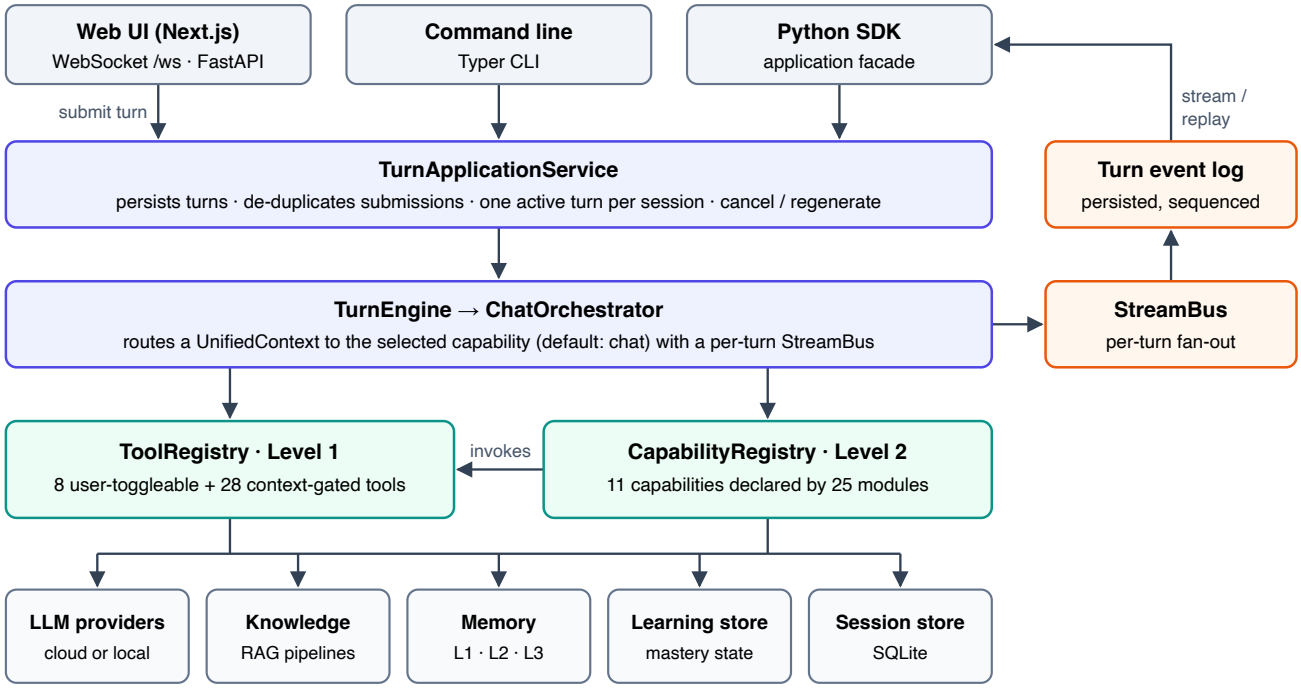


Fig. 1. Runtime architecture. Requests flow down the centre column; streaming events flow back up the right column. Counts are taken from the code base (Section 7).

3. System Architecture

3.1 Entry points and the turn service

The system has three entry points (Fig. 1): a command-line interface built with Typer, a unified WebSocket endpoint served by FastAPI and used by the Next.js web interface, and a Python SDK facade. All three submit work to a single `TurnApplicationService`. A *turn* is one learner message together with everything the system does in response. The service persists each turn before it runs. It computes a SHA-256 digest of the submitted payload, excluding session and workspace identifiers, so that a repeated submission is recognised. A runtime coordinator allows at most one active turn per session. The service also supports cancellation, regeneration of the last turn and answers to questions the tutor asked mid-turn. Clients subscribe to a turn's events from any sequence number, so a browser that reconnects can resume a running turn without losing output.

3.2 Turn engine and orchestrator

The `TurnEngine` constructs a `ChatOrchestrator` lazily, which keeps model providers and plugins out of process start-up, and runs it inside a memory-reclamation scope. For each turn the orchestrator (i) assigns a session identifier if none was given; (ii) checks that the knowledge module is installed when knowledge bases are selected; (iii) chooses the capability: a hand-off capability when a partner or connected agent is to answer, otherwise the capability the learner selected, defaulting to *chat*; (iv) runs it with a fresh `StreamBus`; and (v) publishes a completion event carrying the capability's output. An unknown capability produces an error event followed by a terminal event rather than an exception, so every client sees a well-formed end to every turn.

3.3 A single context object

Every entry point is normalised to one dataclass, `UnifiedContext`. It carries the session identifier, the learner's message, the conversation history in the OpenAI message format, the enabled tools, the selected capability, the selected knowledge bases, attachments, per-request configuration overrides and the response language, together with runtime hooks such as a callback that waits for the learner's reply. Because the orchestrator only ever sees this object, the same capability code serves the terminal, the browser and the SDK.

3.4 Two-layer plugin model

Level 1, tools. A tool subclasses `BaseTool`. It exposes a `ToolDefinition` that converts to an OpenAI function-calling schema, an asynchronous `execute` method returning a `ToolResult`, and prompt hints in each supported language that tell the model when the tool is useful. Eight tools are switched on or off by the learner (Table III). A further 28 are mounted automatically when the context calls for them: for example `rag` when a knowledge base is attached, memory and notebook tools, `ask_user` for questions back to the learner, a sandboxed `exec`, `web_fetch`, reminders and mastery navigation.

Level 2, capabilities. A capability subclasses `TurnCapability`. It declares a `CapabilityManifest` (name, description, stages, tools used, command-line aliases, request schema and configuration defaults) and implements one asynchronous `run(context, stream)` method. Capabilities are declared in 25 module manifests and imported only when used, so an optional module can be disabled without affecting the core. All capabilities report their result through one helper, `emit_capability_result`, so every turn ends with the same envelope: the response payload and a cost summary from a usage tracker.

TABLE I. STREAM EVENT TYPES

Purpose	Event types
Lifecycle	`session, session_meta, stage_start, stage_end, done`
Reasoning and progress	`thinking, observation, progress`
Tool use	`tool_call, tool_result`
Output	`content, sources, result`
Control and errors	`wait_for_input, error`

3.5 Streaming event protocol

Everything a turn produces is a `StreamEvent`: a type (Table I), the source tool or capability, the current stage, a text payload, structured metadata, the session and turn identifiers, a monotonically increasing sequence number and a timestamp. A capability wraps each pipeline stage in an asynchronous context manager that emits paired `stage_start` and `stage_end` events. The orchestrator emits; the turn runtime persists; adapters render, as Rich output in the terminal, React components in the browser or JSON for the SDK (Fig. 2). Because production, persistence and presentation are separate, replay after a reconnection works the same way for every client.

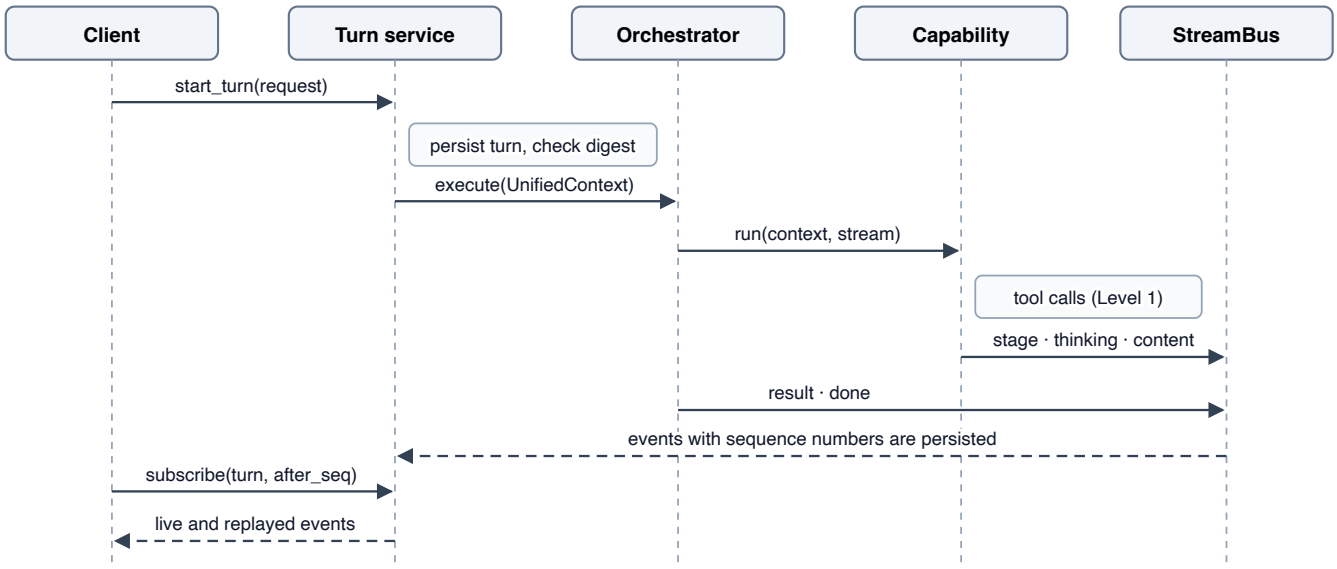


Fig. 2. Lifecycle of one turn. A client that disconnects can resubscribe from its last sequence number and receives the remaining events without loss.

4. CAPABILITIES

Table II lists the eleven built-in capabilities and their stages. *Chat* is the default: an agent loop that may call tools while exploring, followed by a separate stage that streams the final answer. *Ask questions* routes the same loop through `ask_user` to fill in missing context first, and *deep solve* adds problem-planning tools to it. *Deep question* generates practice items in template batches. *Deep research* rephrases the question, decomposes it into subtopics, researches each with tools and writes an iterative report. *Visualize* analyses the request, generates an SVG, Chart.js, Mermaid or HTML artefact and reviews it, or hands off to Manim stages for anima-

tion; *math animator* includes a code-retry stage that repairs Manim code that fails to render. *Video lesson* writes a Remotion lesson with narration that plays in the browser. *Mastery path* drives Guided Learning (Section 5.3). *Immersive reading* reads a document or a video with the learner and cites the page or timestamp behind each answer, and *audio overview* turns a knowledge base into a two-voice audio summary.

TABLE II. BUILT-IN CAPABILITIES

Capability	Module	Stages
chat	core	exploring → responding
ask_questions	core	responding (loop forced through ask_user)
deep_solve	solve	responding (loop with solve-planning tools)
deep_question	questions	ideation → generation
deep_research	research	rephrasing → decomposing → researching → reporting
visualize	visualize	analyzing → generating → reviewing (or Manim stages)
math_animator	renderer-manim	concept_analysis → concept_design → code_generation → code_retry → summary → render_output
video_lesson	video-lesson	code_generation → render_check → render_output
mastery_path	mastery	responding (loop with mastery tools)
immersive_reading	reading	responding (document- or timestamp-grounded)
audio_overview	audio-overview	retrieving → script_writing → voice_generation → publishing

TABLE III. TOOLS THE LEARNER CAN SWITCH ON

Tool	Purpose
brainstorm	Breadth-first idea exploration
web_search	Web search with citations
paper_search	arXiv preprint search
zotero_search	Search a Zotero library
reason	Dedicated deep-reasoning call
geogebra_analysis	Math images to GeoGebra commands
imagegen	Image generation
videogen	Video generation

5. KNOWLEDGE, MEMORY AND MASTERY

5.1 Knowledge bases

Learners build knowledge bases from their own documents. Parsing engines (Docling, MarkItDown, PyMuPDF4LLM, LiteParse and MinerU) convert files to text, and indexing and retrieval are provided by pluggable pipelines: LlamaIndex, LightRAG [9], GraphRAG and PageIndex for local indexes, and connectors to Kiwix, WeKnora and Tencent IMA for collections that live elsewhere. When a knowledge base is attached to a turn, the `rag` tool is mounted and retrieved passages are reported to the client as `sources` events, so answers can be traced to the learner's material.

5.2 Three-layer memory

Memory is organised in three layers. Layer 1 is an append-only trace, one JSON-lines file per surface (chat, book, and so on) per day. Layer 2 holds one Markdown summary per surface. Layer 3 holds cross-surface documents in four slots: *recent*, *profile*, *scope* and *preferences*. A consolidator distils traces into the higher layers with LLM prompts for updating, de-duplicating and auditing them, and the agent reads and writes memory through dedicated tools. Because every layer above the trace is plain Markdown, the learner can inspect and correct what the system believes about them, which addresses part of the privacy and transparency concerns raised in [10].

5.3 Mastery estimation

Guided Learning tracks a mastery score $m \in [0, 1]$ for each knowledge point. Let $c_1, \ldots, c_k$ be the outcomes of the last $k \leq 5$ attempts, oldest first, each in $[0, 1]$ so that partial credit is possible, and let $\mathbf{w} = (0.5, 0.7, 0.85, 0.95, 1.0)$ be recency weights aligned so that the newest attempt has weight 1.0. Then

$$m = \min\left(\frac{\sum_{i=1}^{k} w_{5-k+i} c_i}{\sum_{i=1}^{k} w_{5-k+i}},\ \kappa(k) \right) \quad (1)$$

where the confidence cap is $\kappa(1) = 0.5$, $\kappa(2) = 0.8$ and $\kappa(k) = 1$ for $k \geq 3$. The cap prevents one or two lucky answers from declaring a point mastered, and the weights reward recovery after early mistakes. Table IV shows values computed with the system's own implementation.

TABLE IV. MASTERY SCORES FROM EQ. (1)

Attempt outcomes (oldest → newest)	m
1	0.50
1, 1	0.80
1, 1, 1	1.00
0, 0, 1, 1, 1	0.70
1, 1, 1, 0, 0	0.51
0.5, 0.5, 0.5	0.50

Compared with BKT [4], the rule has no parameters to fit and is easy to explain to a learner, but it does not model guessing or slipping, and it cannot share evidence between related skills. The implementation isolates the rule in a single function, so a fitted BKT or item-response model could replace it without changing any caller.

6. Adaptation for Local Deployment

6.1 Requirements

We set four requirements: (R1) the system runs from a source checkout on one machine, with no container runtime; (R2) it installs and starts on Windows as well as macOS; (R3) it has a distinct product identity while keeping the licence and attribution that Apache-2.0 requires; and (R4) no tested behaviour regresses.

6.2 Repository reduction

We removed files that a local installation never uses (Table V). The rule was conservative: a file was removed only if nothing in the backend, the command line, the frontend source or the build configuration referred to it, which we checked with `git grep` before each deletion. Tests that existed solely to check removed files were removed too: 54 tests in seven files, plus three assertions that read the deleted Dockerfile and documentation. A plugin example that the test suite still loads was moved into the test fixtures rather than deleted. Every module, capability, tool and interface language was kept, along with the experimental TypeScript chat engine and the memory-evaluation harness. The number of tracked files fell from 4,530 to 4,229, including the five files this work adds.

Table V. Files removed, by category

Category	Files
README assets, translations, release notes	176
Electron desktop application	51
Developer handbook	27
CI workflows and issue templates	16
Community and agent documents, hooks	10
Docker and Compose deployment	9
Repository maintenance scripts	7
Tests of removed infrastructure	7
Compatibility wheel	3
Total	**306**

6.3 Renaming the product

The web interface uses natural-language translation keys: the English sentence is itself the key, as in `t("Uploaded to DeepTutor")`. Changing only the displayed values would leave keys out of step with the source, and changing only the source would break look-ups in the ten other languages. We therefore applied one regular expression, `(?<![/\w-])DeepTutor\b`, to keys and values in all eleven catalogues and to string literals in the frontend source and tests (1,755 replacements in 78 files). Every key stays matched, and URLs such as `HKUDS/DeepTutor` are untouched. One sentence that describes the upstream update channel was afterwards restored to name DeepTutor, because the version check still consults upstream releases.

In the backend we renamed the product in prompt templates (84 replacements in 62 files), in user-facing Python string literals (157 in 64 files) and in command-line help (10 in 7 files). Docstrings and identifiers were excluded: the brand followed by "/", "-" or "::" (User-Agent strings, header-style identifiers and Anki deck prefixes), the OAuth client name and the Codex originator. On review we also reverted two header values sent to model providers, so that upstream attribution with those providers is unchanged. The assistant's persona now reads "You are LearnLoop, … built on the open-source DeepTutor project", so the assistant answers honestly when asked who built it. New logo artwork was generated and passed through the repository's existing asset pipeline, which produces every web size and favicon. The internal package name `deeptutor`, with about 20,000 references in 2,400 files, was deliberately kept: renaming it has no user-visible benefit and the highest risk of regression. A `learnloop` command was added as an alias. The licence, copyright notices, citation file and contributor list are retained, and a new NOTICE file credits DeepTutor and lists the changes, as Apache-2.0 §4 requires.

6.4 Windows setup

We added two batch files. `setup.bat` finds Python 3.11–3.14 through the Windows `py` launcher (preferring 3.12), checks for Node.js 22 or newer, creates a virtual environment, installs the Python requirements (which add `pywin32` on Windows through an environment marker) and the package in editable mode, and installs the web dependencies with `npm ci --legacy-peer-deps`, a flag the launcher's own fallback install omits. `start.bat` runs `learnloop start`, which builds the Next.js production bundle on first run and then supervises the backend and frontend. The launcher already starts child processes in their own Windows process groups so that they can be shut down cleanly. Batch files are stored with CRLF line endings through a `.gitattributes` rule, since `cmd.exe` can mis-parse labels in files with LF endings.

6.5 Data and network exposure

All learner data, including API keys, chat history, knowledge bases and memory, lives under `data/` on the local disk. For fully offline use the system can call a local model server (Ollama, LM Studio, llama.cpp or vLLM). The servers listen on all network interfaces and login is disabled by default, so on a shared network such as campus Wi-Fi the operating-system firewall is the only boundary; the README therefore tells users to decline firewall access on public networks.

7. Evaluation

7.1 Method

We evaluated requirement R4 by regression testing. The baseline is the unmodified upstream 1.7.2 checkout; the adapted tree is LearnLoop. Both were tested on the same machine with the same commands: an Apple M3 Pro with 18 GB of memory, macOS 26.6.2, Python 3.13.7, Node.js 24.18.0 and Chrome 157. The Python suite (the `tests/` directory and in-package learning tests) ran under pytest with eight parallel workers. The frontend checks were contract

generation, dependency-architecture rules, type checking, the Node and Vitest suites, lint, translation parity and audit, and a production build.

TABLE VI. REGRESSION RESULTS

Check	Upstream	LearnLoop
Python tests passed	12,688	12,631
Python tests failed*	19	19
Python tests skipped	124	124
Vitest tests passed	1,212 / 1,212	1,212 / 1,212
Node tests passed	1,349 / 1,349	1,348 / 1,348
Type check	pass	pass
Lint errors (warnings)	0 (58)	0 (58)
Architecture rules (1,081 modules)	pass	pass
Contracts, i18n parity and audit	pass	pass
Production web build	pass	pass

* The same 19 tests in both trees (module-lifecycle and migration tests); see Section 7.2.

7.2 Results

Table VI summarises the results. The adapted tree introduced no new test failure. The 57 fewer passing Python tests are exactly those removed with the infrastructure they checked (Section 6.2): 54 in seven deleted files and three trimmed assertions. The one fewer Node test checked the deleted Dockerfile. The 19 failures are identical in both trees and environmental: an instance of the application was running on the test machine and held the lock that guards changes to installed modules, so module-lifecycle tests stopped with "Environment is in use". Re-running them serially in the upstream tree reproduced the same failures. Tests that asserted the old product name failed as expected after the rename; we updated those assertions to the new user-facing text. Two Vitest cases failed once while the Python suite ran in parallel and passed on re-run, which we attribute to timing under CPU contention.

TABLE VII. CODE BASE AFTER ADAPTATION

Component	Files	Lines
Backend Python (excl. tests)	1,127	313,836
Command-line Python	24	6,685
Frontend TypeScript (excl. tests)	1,053	227,694
Prompt templates	201	8,648
TypeScript chat engine	29	5,185
Python tests	1,009	242,026
Frontend tests	468	61,611

Lines include comments and blank lines; generated API contracts are excluded.

7.3 Start-up cost

We measured cold start-up as the median of five runs on the machine described above (Table VIII). The command line starts in under half a second because providers and plugins are imported lazily; importing the whole backend application takes about one second.

TABLE VIII. START-UP TIME, MEDIAN OF 5 RUNS

Operation	Median	Range
`learnloop --help`	0.37 s	0.36–0.55 s
Import backend application	1.01 s	0.98–1.08 s
Import orchestrator only	0.14 s	0.14–0.14 s

7.4 End-to-end check

In a browser we confirmed that the interface shows the new name and logo, and that the About page shows the DeepTutor credit with its licence. We then ran one real conversation through the command line with a cloud model (GPT-4.1) and asked the assistant its name and origin. The turn completed with the expected event sequence (session, stage, progress, 27 content events, result, done), and the assistant answered: "My name is LearnLoop, and I am based on the open-source DeepTutor project developed by the Data Intelligence Lab at HKU."

8. LIMITATIONS AND FUTURE WORK

No learner study. This report evaluates software behaviour, not learning. We propose a study with two groups of students working on one course topic, one using LearnLoop with the course material as a knowledge base and one using the same material with a general-purpose chatbot. Pre- and post-tests would give a normalised learning gain, the System Usability Scale would measure usability, and interaction logs would show which capabilities students use. Such a study needs ethics approval and informed consent; a local deployment helps, since each participant's data stays on their own machine.

Windows validated by review. The batch files and the launcher's Windows code paths were reviewed but, at the time of writing, not executed on Windows hardware.

Model dependence. Answer quality, cost and latency depend on the chosen model. Retrieval reduces but does not remove fabricated content, and a tutor that answers too readily can encourage over-reliance [10].

Security. The network exposure described in Section 6.5 should be addressed by binding to the loopback interface by default or by enabling login.

Learner model. Eq. (1) is transparent but simple; replacing it with a fitted BKT model [4] and comparing the two on logged data is a natural next step.

Upstream drift. A fork does not receive upstream fixes automatically. Because internal package names were kept, upstream changes can still be merged with few conflicts.

9. CONCLUSION

We analysed the architecture of DeepTutor, an open-source agentic tutoring system, and adapted it into LearnLoop, a local installation for Windows and macOS with its own identity. A durable turn service, a two-layer plugin model and a typed event protocol let one code path serve three kinds of client, while retrieval, layered memory and a simple mastery

model support personalised tutoring. A conservative reduction of 306 files, a key-consistent rename and one-step setup scripts made the system easy to install, and regression testing against the upstream release found no change in tested behaviour. The main open question is educational rather than technical: whether learners actually learn more with it, which the proposed study is designed to answer.

Acknowledgments

LearnLoop is a modified version of DeepTutor. We thank its authors, B. Zhao, J. Zhang, X. Ren, Z. Guo, T. Chu, Y. Ma and C. Huang, and the Data Intelligence Lab at The University of Hong Kong for releasing it under the Apache License 2.0; Sections 3–5 describe their design. Portions of the adaptation and of this report were prepared with the help of an AI coding assistant (Claude). All measurements were obtained by running the software.

References

1. B. Zhao, J. Zhang, X. Ren, Z. Guo, T. Chu, Y. Ma, and C. Huang, "DeepTutor: Towards agentic personalized tutoring," arXiv:2604.26962, 2026.
2. B. S. Bloom, "The 2 sigma problem: The search for methods of group instruction as effective as one-to-one tutoring," *Educational Researcher*, vol. 13, no. 6, pp. 4–16, 1984.
3. K. VanLehn, "The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems," *Educational Psychologist*, vol. 46, no. 4, pp. 197–221, 2011.
4. A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User Modeling and User-Adapted Interaction*, vol. 4, no. 4, pp. 253–278, 1994.
5. P. Lewis *et al.*, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, 2020.
6. J. Wei *et al.*, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, 2022.
7. S. Yao *et al.*, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations*, 2023.
8. T. Schick *et al.*, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, 2023.
9. Z. Guo, L. Xia, Y. Yu, T. Ao, and C. Huang, "LightRAG: Simple and fast retrieval-augmented generation," arXiv:2410.05779, 2024.
10. E. Kasneci *et al.*, "ChatGPT for good? On opportunities and challenges of large language models for education," *Learning and Individual Differences*, vol. 103, art. 102274, 2023.